

FINAL IA

Índex

1. Sistemes Basats en el Coneixement (SBC).....	3
1.1. Disseny de l'ontologia.....	3
1.2. Classificació Heurística.....	3
2. Planificació Automàtica.....	4
2.1. Modelat del Domini.....	4
2.2. Modelat del Problema (Instància).....	4

Els exàmens finals de IA es solen estructurar típicament en dos grans blocs de 5 punts cadascun: Sistemes Basats en el Coneixement (SBC), centrat en el disseny d'ontologies i classificació heurística; i Planificació Automàtica, centrat en el modelat de dominis i problemes en llenguatge PDDL.

1. Sistemes Basats en el Coneixement (SBC)

Aquesta part evalua la capacitat per a estructurar informació complexa i dissenyar un sistema que prengui decisions expertes. Es divideix en dos apartats:

1.1. Disseny de l'ontologia

L'objectiu és extreure els conceptes de l'enunciat i relacionar-los entre ells.

- **Identificar problema i solució:** subrallar els elements que són les dades d'entrada (Problema Concret) i els que formen la sortida que el sistema ha de proporcionar (solució).
- **Creació de Jerarquies (Taxonomia):** s'han d'agrupar conceptes comuns en superclasses per a evitar redundàncies.
- **Regla per a les subclasses:** només s'han de crear subclasses si aquestes tenen atributs o relacions diferents entre sí. Si comparteixen exactament els mateixos atributs i relacions, es millor utilitzar un atribut de tipus (enumeració) en la classe pare. Però, es valor positivament que branques germanes de la taxonomia tinguin un nivell de granularitat similar.
- **Atributs abstractes:** és fonamental incloure en l'ontologia les característiques abstractes que es mencionen en el text (ex: pressupost baix, nivell de risc alt). D'aquesta manera tots els conceptes que s'utilitzaran en les regles del següent apartat estaran soportades per l'ontologia.
- **Llistar atributs:** per a cada concepte, llista els seus atributs indicant el nom, tipus, i, si és una enumeració, els valors exactes que pot pendre.

1.2. Classificació Heurística

S'ha de justificar la metodologia i crear regles que simulin un raonament humà en tres fases.

1. Justificar el tipus de la solució

S'ha d'indicar si la solució demanada requereix refinament (solució concreta) o no (solució abstracte). Requereix de refinament si es necessita accedir a dades específiques del problema concret (com un pressupost exacte en euros, o codi postal) per a calcular o generar la sortida final (ex: trucar a una funció per a llistar resultats).

2. Definir els 4 vocabularis

- Problema Concret: les dades brutes d'entrada (ex: m² de la casa, salari de l'usuari).
- Problema Abstracte: característiques qualitatives derivades de les dades concretes (ex: pressupost = baix).
- Solució Abstracte: les categories de la solució (ex: tipus vehicle recomenat = todoterreno).

- Solució Concreta: la sortida específica (ex: preuTaxació = 120.000).

3. Escriure les regles (mínim 3 de cada tipus)

- Regles d'Abstracció: transformen dades concretes en abstractes. Exemple:
si Usuari.pressupost < 10000 euros llavors Usuari.Tipus_Pressupost = baix
- Regles d'Associació Heurística: relacionar el problema abstracte amb la solució abstracta. Exemple:
si Usuari.Tipus_Pressupost = baix i Usuari.Terreny = off-road llavors
Recomenació.Tipus_Vehicle = todoterreno
- Regles de Refinament: usen la solució abstracta i dades del problema concret per a donar la resposta final. Exemple:
si Recomendació.Tipus_Aquisició = compra llavors
mostrar_llista_vehicles(Recomenació.Tipus_Vehicle, Usuari.pressupost_total)

2. Planificació Automàtica

Aquesta part evalua la capacitat per a traduir un problema de logística, creació o moviment a un llenguatge de planificació clàssic.

2.1. Modelat del Domini

- **Definir requirements**: generalment es necessita :strips :typing, :adl (per a quantificadors i condicionals).
- **Tipus (:types)**: utilitzar jerarquies de tipus i subtipus és vital per a guanyar eficiència. Redueix dràsticament el nombre d'unificacions que el planificador ha de fer.
- **Predicats (:predicates)**: agrupa lògicament els predicats (moviments, estats dels objectes, etc...).
- **Accions (:action)**:
 - Ordre restrictiu: posar sempre en la precondició les condicions més restrictives (les que unifiquen menys objectes) per a accelerar el càlcul.
 - Evitar colls d'ampolla: no abusar de forall i exists en les precondicions, ja que penalitzen greument al temps de còmput i a la nota. Si és possible, s'ha d'utilitzar predicats guia o de filtre per a evitar buscar entre tots els objectes.
 - Control de recursos: si dos robots o màquines no poden utilitzar el mateix ítem a la vegada, hem d'afegir un predicat com (libre ?ítem) o modelar-ho de forma que l'acció els bloquegi (not (libre ?ítem)) per a evitar conflictes lògics irrealis.
 - Funcions matemàtiques: si hi ha distàncies, temps de fabricació o costos, utilitzar comandes matemàtiques en l'efecte (increase, decrease, assign) operant sobre les funcions declarades.

2.2. Modelat del Problema (Instància)

- Objectes (:objectes): llista tots els elements que participen en l'escenari descrit separant-los per els tipus definits al domini.
- Estat inicial (:init):

- Definir les posicions inicials, les connexions del mapa (ej: (accessible Casa Bosque))
- Iniciar a 0 (o valor corresponent) les funcions numèriques o fluents ((= (temps) 0))
- Objectiu (:goal): definir quina condició s'ha de complir per a terminar. En comptes d'escriure l'estat final de tots els objectes un a un, aquí és molt útil utilitzar forall (ex: (forall (?p -pedido) (entregado ?p))), ja que el cost és mínim en l'etapa meta i permet que el codi escalí sense problemes.
- Optimització (:metric): si l'enunciat demana minimitzar el temps, recursos o cost, afegir la instrucció final (ex: (:metric minimize (temps))).